

Skill-Design Lessons for Multi-Skill Agent Pipelines

If you own a skill in a multi-skill pipeline, the patterns below are worth applying preemptively.

1. Anchor turn-role at the TOP of a skill body, not the bottom

For every non-terminal skill, add a top-of-file banner such as `## NON-TERMINAL – same-turn handoff required` as the **first section** below the front-matter. Models compose output sequentially — if a handoff instruction only appears after hundreds of lines of skill logic, the model has already committed to an answer-shape by the time it reaches the instruction.

Example. In one variant, the "your output pairs with a tool_call in the same response" statement lived in a `## MANDATORY NEXT STEP` block at the bottom of each skill file (~350 lines in). Moving the same statement to a top-of-file banner in a second variant reduced silent-stop occurrences on those skill boundaries meaningfully. The information was already there; only the position changed.

Rule of thumb: if your skill is non-terminal, the *first* thing the model reads after the description must state that its output pairs with a tool call in the same response.

2. Commit the handoff token as the FIRST key of the output

Put a routing marker such as `_pipelineNext: "<next-skill>"` as the leading key of every wrapped output schema. This is a self-priming anchor: the routing decision is committed to the token stream *before* the payload body is written. It prevents the "silent stop after emitting valid body" class where the model produces a full answer, then forgets to fire the next tool_call and halts.

Example. The silent-stop halt class is remarkably resistant to prose. Every "empty content is FORBIDDEN", "you MUST fire the next tool_call", "do not stop here" reminder tested had near-zero effect on the halt rate — variants stayed within measurement noise. The structural fix — moving the routing key from anywhere in the output to the mandatory first key — dropped the class by 5–10 percentage points on its own. Prose asks the model to remember something; the schema requires it.

Rule of thumb: the FIRST key the model writes should encode the pipeline hop, not the semantic payload.

3. Centralize routing knowledge in ONE place; delegate from skill bodies

Give the router (main system prompt / maker instructions) a single **HANDOFF MAP** — a table listing every transition and the arguments to pass. Skills should reference the map by name, not restate it inline.

Example. One variant had each of 6 non-terminal skills carrying its own ~30-line "MANDATORY NEXT STEP" block that restated argument contracts. Six copies of nearly-identical information, all slightly drifted from each other after multiple edits. The variant that replaced these with a single central HANDOFF MAP in the router + 3–5-line "NEXT (same response)" footers per skill gained ~3–5 pass-rate points. Fewer copies of the same information means less drift, less priming surface, and one place to change when the pipeline evolves.

Rule of thumb: one authoritative table for cross-skill handoffs; skill bodies reference, never restate.

4. Do NOT teach the router the classification rule a skill performs

If a routing skill's job is to decide between two flow types based on input shape, the router prompt must **not** contain a paragraph explaining the same decision rule.

Example. One variant of the router included a paragraph explaining the classification rule inline: *"if input has keys X, Y, Z (all non-empty strings, exact case) then this is flow-A; otherwise this is flow-B"*. The model read the rule, executed it inline, and skipped the classifier skill entirely — jumping straight to the second skill in the pipeline. This produced a specific failure class where the pipeline ran N-1 skills instead of N. Deleting that paragraph — so the router only says "invoke the classifier first" — eliminated the class.

More generally: **the router describes *what skills exist*, not *what they compute***. If the router teaches the model to compute the answer, the router just made the skill optional.

Rule of thumb: router prose should be limited to "invoke skill X when Y" — never "if condition Y, then compute Z" for anything a downstream skill produces.

5. Post-tool-result silent-stop needs explicit, in-skill guards

Every skill that fans out to a sub-tool (e.g. a search or a data-lookup tool) is at risk of the "empty content after tool result" halt. Global rules in the router do not prevent this reliably. Each such skill needs its own

"Silent-stop guard (post-tool-result)" section stating what the model must produce on the turn *immediately after* the tool result returns.

Example. A skill invoking a search tool consistently halted on 30–50% of runs after the search results returned — the model would receive the results and then emit `finish_reason=stop` with empty content. Global router guards did not prevent this. Adding a per-skill "post-tool-result guard" at the top of that specific skill's file — *"The moment the search results return, your NEXT response MUST be: [wrapped JSON] plus the next skill invocation"* — reduced the halt rate on that boundary by roughly half. The guard has to sit inside the skill where the risk originates.

Rule of thumb: guard the specific boundary where a failure class fires (post-tool-result of THIS skill), not the general concept elsewhere.

6. Prefer STRUCTURAL markers over IMPERATIVE prose (*the largest single lever*)

Imperative phrasing (`You MUST invoke ...`, `Do NOT stop here`, `The next skill IS ...`) can be ignored during generation. Structural constraints — top-of-file banners, ordered keys, table lookups, section positions — shape *what tokens get emitted next* and are harder to bypass.

Example. The most dramatic result in the entire study. One router variant described the pipeline as a turn-by-turn wire template:

Turn 1	<code>tool_calls=[skill(A)]</code>	<code>content=None</code>
Turn 2	<code>tool_calls=[skill(B)]</code>	<code>content=<A's JSON></code>
Turn 3	<code>tool_calls=[skill(C)]</code>	<code>content=<B's JSON></code>
Turn 4	<code>tool_calls=[KnowledgeSearch x3]</code>	<code>content=None</code>
Turn 5	<code>tool_calls=[skill(D)]</code>	<code>content=<C's JSON></code>

The intent was to give the model a "turn-by-turn contract". The result: **the model reproduced `{"tool_calls": [...]}` as chat content on turn 1 in ~85% of runs**, halting the pipeline immediately. The wire-shape template was interpreted as an output specification the model should reproduce. Rewriting the same information as pipeline-position prose:

Turn 1	-> A	(router seed; no upstream JSON exists yet)
Turn 2	-> B	upstream: A's JSON
Turn 3	-> C	upstream: B's JSON
Turn 4	-> KnowledgeSearch x3 parallel	required by C's body
Turn 5	-> D	upstream: C's JSON

... dropped that failure class from ~85% to under 2%. About 80 pass-rate points of improvement from removing wire-shape syntax alone. This was the single largest lever observed in the entire study.

Rewrite each imperative you're tempted to add as a structural requirement instead: instead of "MUST call the next skill", make the next-skill name a required first key of the output schema.

Rule of thumb: if a passage describes shape (`content=X`, `tool_calls=[Y]`), the model will reproduce that shape. Rewrite as pipeline position or role.

7. Positive framing beats "don't do this" examples

Naming the wrong shape as a forbidden example puts that shape in the model's context as a token sequence it can reproduce. Every occurrence of the leak-shape example in the prompt lifts the probability that the model emits it.

Example. A router variant contained:

Do NOT serialize `skill(...)`, `{"name":"skill",...}`, or any tool-call JSON into the content field.

The forbidden shape (`{"name":"skill",...}`) is itself a token sequence the model can copy. Rewriting to positive-only framing —

All skill invocations flow through the platform mechanism; nothing about tool invocation belongs in the content field.

— dropped that specific mimicry class by 1–2 percentage points. Same behavioral constraint, no template to copy.

The same lesson applies to specific input tokens: if you're tempted to write *"the classifier tends to skip on inputs like 'ACME Foo Corp'"*, **don't**. Use `<placeholder>` tokens; naming specific failing inputs near failure prose creates priming.

Rule of thumb: state what SHOULD happen; do not name the wrong shape you're trying to prevent.

8. Sibling consistency is a first-order concern

When skills in the same pipeline family use different output schemas, different footer styles, or different naming conventions, the model gets conflicting cues about what "correct" output looks like. When they share structure (all non-terminal wrapped-output skills start with the same routing key, all non-terminals

share the same top-of-file banner; all NEXT footers follow the same layout), the model generalizes correctly to new skills of the same class.

Example. In one iteration, four skills of the same architectural role each used slightly different output wrappers — one had `{"data": {...}}`, one had `{"result": {...}}`, one had a flat structure, one had `{"<skillName>": {...}}`. Even though each individually was internally correct, the model's generalization from one skill to the next was noisy — pass rates varied wildly by which skill was invoked. Refactoring all four to use the identical `{"_pipelineNext": "...", "<skillName>": {...}}` wrapper eliminated the variance.

Rule of thumb: if two skills play the same architectural role, their skill-definition files should be nearly copy-paste identical apart from domain content.

Applying this to a new skill

Before checking in a new skill definition, verify:

1. Does it start with a non-terminal / terminal banner appropriate to its role?
2. If non-terminal wrapped, is the routing/handoff key the FIRST key of the output schema?
3. Is its NEXT footer compact (5–10 lines), not a full argument-contract restatement?
4. Does the router's central HANDOFF MAP have the row(s) for this skill's transitions?
5. Does its structure match its sibling skills of the same architectural role?
6. If it invokes a sub-tool, does it have a post-tool-result silent-stop guard?
7. Does the router prose describe *when* to call this skill, but NOT *what it computes*?
8. Are any prose examples using `<placeholder>` tokens rather than real failing inputs?
9. Are any "don't do this" examples rewritten as positive framing (no leak-shape templates in prose)?

Answering "no" to any of these predicts a specific, previously-observed failure mode.